

Apunte de la materia, de la primera variable a la máquina de estados

C para software de vuelo

Ingeniería de Software de Vuelo para Sistemas Espaciales Críticos · 2.º cuatrimestre 2026

v0.3 — módulos 1 a 4 de 12

Para qué es esto. Para aprender el C que usa la materia, en el orden en que lo da la cátedra, con cada programa **compilado de verdad** con los mismos flags que pide el práctico. Si un ejemplo de acá no compila, el apunte no se publica: lo chequea un script, no la buena voluntad.

Lo que no es. No resuelve los prácticos: los ejemplos están ambientados en el mismo OBC imaginario, pero con otros números y otros casos. Copiar y pegar no va a andar, y ésa es la idea.

El código sale en verde con su salida abajo, en negro, copiada de la corrida real en el gcc de Ubuntu: lo que ves impreso es lo que imprimió.

Los colores no son adorno: cada uno dice una cosa y sólo una.

Azul	en el texto, La idea : — lo que no hay que perder.
Rojo	en el texto, Ojo : — donde se traba casi todo el mundo.
Violeta	lo que pide la cátedra, sacado de sus clases y de sus TP.
Verde azulado	una mejora : algo que agregamos nosotros y no es de la cátedra. Se puede saltar.
Rosa	la posta: lo mismo, dicho en criollo y sin vueltas.

Índice

1 El programa mínimo	1
1.1 La plantilla de la cátedra	1
1.2 Compilar: de texto a programa	2
1.3 Leer un warning	2
1.4 Un programa de vuelo no termina	3
2 Variables y tipos	5
2.1 Qué es una variable	5
2.2 Los tipos de C, y por qué no alcanzan	5
2.3 La solución: tipos de ancho fijo	6
2.4 Con signo o sin signo: dos preguntas	6
2.5 Imprimir con el especificador correcto	7
2.6 La primera trampa: el desborde	7
2.7 La segunda trampa: la división entera	8
3 Constantes y calificadores	10
3.1 Los literales: cómo se escribe un valor	10
3.2 #define : un nombre para un número	11
3.3 La trampa del #define : los paréntesis	12
3.4 const : el dato que no se toca	13

3.5	<code>volatile</code> : lo que cambia por fuera	13
3.6	<code>static</code> : dos palabras con el mismo nombre	15
4	Operadores, y los de bit	17
4.1	Los aritméticos, y el resto	17
4.2	Relacionales y lógicos	18
4.3	El ternario	19
4.4	Quién va primero: la precedencia	19
4.5	Los operadores de bit	21

El programa mínimo

AL TERMINAR ESTE MÓDULO VAS A PODER

Escribir un programa en C con la estructura que pide la cátedra, compilarlo con los flags de la materia, leer un warning sin entrar en pánico, y explicar por qué un programa de un satélite no termina nunca.

C nació en 1972 en los laboratorios Bell, y medio siglo después sigue siendo el idioma en que se escribe casi todo el software que vuela. No es nostalgia. En un satélite el procesador es chico, la memoria es poca y nadie va a subir a reiniciarlo, así que hace falta un lenguaje que haga **exactamente** lo que se le escribe: nada de recolector de basura que se despierta cuando quiere, nada de magia escondida. C traduce casi uno a uno a lo que el procesador ejecuta, deja tocar el hardware directo y se comporta igual todas las veces. La contracara es que tampoco te cuida: si le pedís algo absurdo, lo hace con toda la cortesía del mundo.

La idea: todo lo que viene en este apunte apunta a lo mismo que la clase de Leandro pone en una diapositiva — escribir código **correcto, predecible y mantenible** para un sistema que no se puede reparar una vez lanzado —. Cada tema de C que aparezca va a tener esa pregunta atrás: ¿esto me ayuda a que el OBC no se cuelgue a 500 km de altura?

1.1 La plantilla de la cátedra

Todos los ejercicios de la materia arrancan de la misma plantilla, `template.c`. No es burocracia: ordena el archivo para que cualquiera —incluido vos, dentro de tres semanas— encuentre las cosas siempre en el mismo lugar. Ésta es la plantilla con lo mínimo adentro:

LA PLANTILLA, CON UN PROGRAMA ADENTRO — `m01-template.c`

```
/* C libraries */
#include <stdio.h>
#include <stdint.h>

/* Macros */
#define MISION "MISION-DEMO"

/* User libraries */

/* Global variables */

/* Functions declaration */

int main(void) {
    /* Main program */
    printf("OBC %s: arranque OK\n", MISION);
    return 0;
}

/* Functions definition */
```

SALIDA (copiada de la corrida real)
OBC MISION-DEMO: arranque OK

De arriba hacia abajo, cada sección tiene un trabajo:

Sección	Qué va
C libraries	Las bibliotecas del sistema, con <code>#include <...></code> . <code>stdio.h</code> trae <code>printf</code> ; <code>stdint.h</code> trae los tipos de ancho fijo del módulo que viene.

Macros	Las constantes con nombre, con <code>#define</code> . Acá, el nombre de la misión.
User libraries	Las bibliotecas propias, con <code>#include "..."</code> (comillas, no ángulos). Aparecen en el módulo 11.
Global variables	Lo que tiene que ver todo el archivo. Cuanto menos, mejor.
Functions declaration	La firma de cada función propia: nombre, qué recibe, qué devuelve. Va arriba de <code>main</code> para que el compilador la conozca antes de que alguien la llame.
main	Donde arranca el programa. Siempre <code>int main(void)</code> .
Functions definition	El cuerpo de cada función declarada arriba.

`int main(void)` se lee así: una función que se llama `main`, no recibe nada (`void`) y devuelve un entero (`int`). Ese entero es el `return 0` del final, y le avisa al sistema operativo que todo salió bien; cualquier otro número es un código de error. En la placa no hay sistema operativo que lo lea, pero en la PC sí, y la terminal lo usa.

Ojo: el punto y coma cierra cada **instrucción**, no cada renglón. Las llaves `{ }` agrupan instrucciones y no llevan punto y coma después. Olvidarse uno es el error número uno de toda la carrera, y el compilador lo va a reportar en la línea **siguiente**, porque recién ahí se da cuenta.

1.2 Compilar: de texto a programa

El procesador no entiende C: entiende instrucciones de máquina. El compilador traduce. En la terminal de Ubuntu, con el comando que pide la materia:

```
$ gcc -Wall -Wextra -std=c11 m01-template.c -o obc
$ ./obc
OBC MISION-DEMO: arranque OK
```

Pedazo	Qué hace
<code>gcc</code>	El compilador de C de GNU, el mismo que trae CubeIDE adentro (en su versión para ARM).
<code>-Wall</code>	Prende los avisos (<i>warnings</i>) más comunes. A pesar del nombre, no son todos .
<code>-Wextra</code>	Prende otro montón que <code>-Wall</code> deja afuera. Entre los dos, el compilador se vuelve un compañero desconfiado, que es lo que se quiere.
<code>-std=c11</code>	Compila según el estándar C de 2011. Así un programa se comporta igual en cualquier compilador que lo respete.
<code>-o obc</code>	El nombre del ejecutable que sale. Sin esto se llama <code>a.out</code> , que no le dice nada a nadie.
<code>./obc</code>	Lo corre. El <code>./</code> es «el que está en esta carpeta».

LO QUE PIDE LA CÁTEDRA — CERO WARNINGS

El práctico lo dice sin anestesia: un ejercicio **no está terminado** hasta que compila con **cero** warnings con esos tres flags. Compilar no alcanza; compilar callado, sí.

1.3 Leer un warning

Un warning es el compilador diciendo «esto es legal, pero me huele mal». El programa se genera igual, y por eso la tentación es ignorarlo. En software de vuelo un warning ignorado es una falla que todavía no ocurrió. Este programa declara una temperatura y nunca la usa:

UNA VARIABLE QUE NADIE USA — `m01-warning.c`

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    int16_t temperatura = -18; /* declarada... y olvidada */
    printf("Reporte enviado\n");
    return 0;
}
```

```
LO QUE DICE GCC (copiado de la corrida real)
m01-warning.c: In function 'main':
m01-warning.c:5:17: warning: unused variable 'temperatura' [-Wunused-variable]
   5 |         int16_t temperatura = -18; /* declarada... y olvidada */
     |         ^~~~~~
```

Se lee en tres partes, y siempre son las mismas. Primero **dónde**: `m01-warning.c:5:17` es archivo, línea 5, columna 17 (gcc cuenta el tabulador como ocho columnas). Después **qué**: `unused variable` (variable sin usar). Al final, entre corchetes, **qué flag lo prendió**: `-Wunused-variable`, que viene incluido en `-Wall`. Abajo, gcc dibuja el renglón y apunta con `^` al lugar exacto, que es más de lo que hacen muchos profesores.

Este warning en particular parece inofensivo, y casi siempre esconde algo peor: si declaraste la temperatura es porque la ibas a mandar en la telemetría, y te olvidaste. El compilador no sabe qué querías hacer, pero sabe que no lo hiciste.

LA POSTA

Un warning es un error que todavía no te mordió. Se leen de arriba hacia abajo, se arregla **el primero** y se vuelve a compilar: muchas veces los de abajo eran consecuencia de ése y desaparecen solos.

1.4 Un programa de vuelo no termina

En la PC, un programa arranca, hace lo suyo y termina. En el OBC no: si el programa terminara, el satélite se quedaría mudo hasta que alguien lo reinicie, y ese alguien está a varios cientos de kilómetros. Por eso todo programa embebido tiene la misma forma: **una inicialización que corre una vez**, y después **un bucle infinito** que repite lo mismo mientras haya energía —leer sensores, atender telecomandos, mandar telemetría—. Se llama *superloop*, y en la placa se escribe `while (1)`.

Para poder mostrarlo acá, el ejemplo lo corta a tres vueltas; en la placa esa cota no existe:

EL SUPERLOOP, RECORTADO PARA LA PC — `m01-superloop.c`

```
#include <stdio.h>
#include <stdint.h>

#define VUELTAS_DE_PRUEBA 3u /* en la PC lo cortamos; en la placa es while (1) */

int main(void) {
    uint32_t vuelta = 0;

    /* Inicializacion: corre UNA sola vez */
    printf("Iniciando perifericos...\n");

    /* Superloop: lo mismo, una y otra vez, mientras haya energia */
    while (vuelta < VUELTAS_DE_PRUEBA) {
        vuelta++;
        printf("Vuelta %u: leer sensores, atender telecomandos, mandar telemetria\n", vuelta);
    }
    return 0;
}
```

```
SALIDA (copiada de la corrida real)
Iniciando periféricos...
Vuelta 1: leer sensores, atender telecomandos, mandar telemetria
Vuelta 2: leer sensores, atender telecomandos, mandar telemetria
Vuelta 3: leer sensores, atender telecomandos, mandar telemetria
```

Ojo: que el bucle **principal** no termine no quiere decir que **cualquier** bucle pueda no terminar. Un **while** que espera a un sensor que se rompió cuelga al OBC entero. Los bucles de espera llevan siempre una cota: se ve en el módulo 5.

MEJORA, NO ES DE LA CÁTEDRA – COMPILAR DESDE VS CODE

El mismo comando se puede atar a una tecla en VS Code (una **tarea**, `tasks.json`), así no se tipea cada vez. La guía de IDEs de la materia explica cómo.

MÓDULO 2

Variables y tipos

AL TERMINAR ESTE MÓDULO VAS A PODER

Elegir el tipo de cada dato contestando dos preguntas —¿puede ser negativo? ¿cuánto rango necesita?—, saber cuánto ocupa en la PC y en la placa, imprimirlo con el especificador correcto, y reconocer las dos trampas que más telemetría arruinan: el desborde y la división entera.

2.1 Qué es una variable

Una variable es un pedazo de memoria con **nombre**. Tiene cuatro cosas, y conviene nombrarlas desde ya porque las cuatro vuelven: un **nombre** (*temperatura*), un **tipo** (*int16_t*: cuántos bits ocupa y cómo se leen), un **valor** (*-18*) y una **dirección**, el lugar de la memoria donde vive. La dirección la vas a ignorar hasta el módulo de punteros, y ese día va a ser la protagonista.

En C toda variable se **declara** antes de usarla, diciendo su tipo. Declarar e **inicializar** (darle valor) se puede hacer junto, y conviene:

```
int16_t temperatura = -18; /* declarada e inicializada */
uint16_t corriente;      /* declarada: vale... cualquier cosa */
```

Ojo: una variable local declarada sin valor **no vale cero**: vale lo que haya quedado en ese pedazo de memoria, que puede ser cualquier cosa y cambiar de una corrida a otra. Es el error que funciona en tu máquina y falla en la de otro. Se inicializa siempre, aunque sea en cero.

2.2 Los tipos de C, y por qué no alcanzan

C trae cuatro tipos básicos: *char* (un carácter, o un entero chico), *int* (entero), *float* (decimal de precisión simple) y *double* (decimal de precisión doble). Se les pueden agregar calificadores de tamaño (*short*, *long*) y de signo (*signed*, *unsigned*). Hasta acá, lo de cualquier curso de C.

El problema es que C **no garantiza** cuánto mide cada uno: lo decide cada compilador según la máquina. El estándar sólo promete mínimos —un *int* tiene **al menos** 16 bits—. Este programa pregunta los tamaños con *sizeof*, que devuelve cuántos bytes ocupa un tipo:

CUÁNTO MIDE CADA TIPO, EN LA PC —m02-sizeof.c

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    printf("char      : %zu byte\n", sizeof(char));
    printf("short     : %zu bytes\n", sizeof(short));
    printf("int        : %zu bytes\n", sizeof(int));
    printf("long       : %zu bytes\n", sizeof(long));
    printf("float      : %zu bytes\n", sizeof(float));
    printf("double     : %zu bytes\n", sizeof(double));
    printf("uint8_t    : %zu byte\n", sizeof(uint8_t));
    printf("int16_t    : %zu bytes\n", sizeof(int16_t));
    printf("uint32_t   : %zu bytes\n", sizeof(uint32_t));
    return 0;
}
```

```

SALIDA (copiada de la corrida real)
char      : 1 byte
short     : 2 bytes
int       : 4 bytes
long      : 8 bytes
float     : 4 bytes
double    : 8 bytes
uint8_t   : 1 byte
int16_t   : 2 bytes
uint32_t  : 4 bytes

```

Ésa es la salida en la PC con Ubuntu. El mismo programa, compilado con el gcc para ARM que trae CubeIDE, da **long** de 4 bytes, no de 8 (medido con el compilador de CubeIDE 1.18.1 para un Cortex-M4, el de la NUCLEO-F446RE). Mismo código, mismo nombre de tipo, la mitad de bits. Si ese **long** era un contador que viaja en un paquete de telemetría, la estación terrena lo va a leer mal y nadie va a entender por qué.

2.3 La solución: tipos de ancho fijo

La biblioteca `<stdint.h>` trae tipos cuyo nombre **dice** cuánto miden, en cualquier máquina:

Tipo	Bits	Rango
<code>uint8_t</code>	8	0 a 255
<code>int8_t</code>	8	-128 a 127
<code>uint16_t</code>	16	0 a 65 535
<code>int16_t</code>	16	-32 768 a 32 767
<code>uint32_t</code>	32	0 a 4 294 967 295
<code>int32_t</code>	32	-2 147 483 648 a 2 147 483 647

El nombre se lee solo: **u** es *unsigned* (sin signo), el número son los bits, y **_t** es «tipo». La regla del rango sale de contar: con n bits hay 2^n combinaciones. Sin signo van de 0 a $2^n - 1$; con signo, la mitad se usa para los negativos, y van de -2^{n-1} a $2^{n-1} - 1$. Por eso `int8_t` llega a 127 y no a 128: el cero ocupa un lugar del lado positivo.

LO QUE PIDE LA CÁTEDRA — ANCHO FIJO, SIEMPRE

La clase lo dice en mayúsculas: en código embebido se usan **siempre** tipos de ancho fijo, y **unsigned** por defecto para lo que son cantidades o patrones de bits. El tamaño tiene que ser inequívoco y explícito.

2.4 Con signo o sin signo: dos preguntas

Para cada dato, antes de escribir el tipo, se contestan dos preguntas: ¿**puede ser negativo?** y ¿**hasta dónde puede llegar?** Una rueda de reacción —la que gira para orientar el satélite— lo ilustra bien:

TELEMETRÍA DE UNA RUEDA DE REACCIÓN – m02-rueda.c

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint16_t muestra = 1207; /* contador de muestras: nunca es negativo */
    int16_t velocidad = -3450; /* rpm: el signo dice para que lado gira */
    float tension = 11.8f; /* V del bus de potencia */
    uint16_t corriente = 420; /* mA */

    /* mA -> A ANTES de multiplicar, o la potencia sale mil veces grande */
    float potencia = tension * (corriente / 1000.0f);

    printf("Muestra #%u\n", muestra);
    printf("Velocidad: %d rpm\n", velocidad);
    printf("Bus: %.1f V, %u mA -> %.2f W\n", tension, corriente, potencia);
    return 0;
}
```

SALIDA (copiada de la corrida real)

```
Muestra #1207
Velocidad: -3450 rpm
Bus: 11.8 V, 420 mA -> 4.96 W
```

El número de muestra no puede ser negativo y crece: `uint16_t`, que llega a 65 535. La velocidad sí puede ser negativa, porque la rueda gira para los dos lados y el signo es justamente **para cuál**: `int16_t`. La corriente en miliampere es una cantidad: `uint16_t`. La tensión tiene decimales: `float`.

Ojo: la corriente viene en **mili**ampere, y la potencia se calcula en watt. Si se multiplica sin convertir, el resultado sale mil veces más grande: el satélite estaría consumiendo 4956 W, más o menos lo de un horno eléctrico con el aire acondicionado prendido. Las unidades se convierten **antes** de operar, y el `1000.0f` (con punto) es a propósito: se ve en la última sección.

2.5 Imprimir con el especificador correcto

`printf` no sabe qué tipo le pasaste: se lo dice el especificador, y si no coinciden imprime cualquier cosa. Los que se usan en la materia:

Especificador	Para	Ejemplo
%d	enteros con signo	-3450
%u	enteros sin signo	1207
%f, %.2f	float y double; .2 son los decimales	4.96
%X, %02X	hexadecimal; 02 rellena con ceros a dos dígitos	0F
%c	un carácter	A
%s	una cadena	MISION-DEMO
%zu	lo que devuelve sizeof	4
%%	el signo de porcentaje mismo	%

Ojo: en la PC, `printf("%u", x)` con un `uint32_t` compila sin quejas. En la placa **no**: para el gcc de ARM, `uint32_t` es un `long unsigned int`, y -Wall avisa que %u espera un `unsigned int` (medido con el compilador de CubeIDE). El arreglo portable es la macro `PRIu32` de `<inttypes.h>`: `printf("%" PRIu32 "\n", x);`. Se ve fea, y funciona en las dos máquinas, que es lo que importa.

2.6 La primera trampa: el desborde

Un tipo tiene un rango, y C no avisa cuando se lo pasa. Un contador de paquetes de 8 bits:

UN CONTADOR DE 8 BITS QUE SE DA VUELTA — m02-desborde.c

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint8_t paquete = 254;    /* 8 bits: de 0 a 255, y ni uno mas */

    for (int i = 0; i < 3; i++) {
        printf("Paquete #%u\n", paquete);
        paquete++;            /* despues de 255 vuelve a 0, sin avisar */
    }
    return 0;
}
```

```
SALIDA (copiada de la corrida real)
Paquete #254
Paquete #255
Paquete #0
```

Después de 255 viene 0. En un tipo **sin signo** eso está definido por el estándar: la cuenta da la vuelta como un cuentakilómetros. Si la estación terrena usa ese número para detectar paquetes perdidos, va a creer que perdió 255 de golpe. El arreglo es elegir el ancho pensando en el **peor caso**: ¿cuántos paquetes manda el satélite en una pasada? ¿Y en un año?

Ojo: en un tipo **con signo** el desborde es todavía peor: el estándar dice que es *comportamiento indefinido*, o sea que el compilador puede hacer lo que quiera, incluido algo distinto según cuánto optimice. No es «da la vuelta a -128»: es «no se sabe». Con signo, se chequea **antes** de sumar.

2.7 La segunda trampa: la división entera

Si los dos operandos de una división son enteros, C hace división **entera**: tira los decimales, sin redondear. Calcular el porcentaje de batería parece una regla de tres, y es un campo minado:

TRES FORMAS DE CALCULAR UN PORCENTAJE — m02-division.c

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint16_t mv = 3790;    /* bateria: 3000 mV es 0 %, 4200 mV es 100 % */

    uint16_t mal = (mv - 3000) / (4200 - 3000) * 100;    /* divide primero: 790/1200 = 0 */
    uint16_t bien = (mv - 3000) * 100 / (4200 - 3000);    /* multiplica primero */
    float exact = (mv - 3000) * 100.0f / (4200 - 3000);    /* un float en la cuenta */

    printf("mal : %u %%\n", mal);
    printf("bien : %u %%\n", bien);
    printf("exact : %.1f %%\n", exact);
    return 0;
}
```

```
SALIDA (copiada de la corrida real)
mal : 0 %
bien : 65 %
exact : 65.8 %
```

La primera divide antes de multiplicar: $790 / 1200$ da 0 (es menos de uno, y la parte decimal se tira), y cero por cien es cero. La batería está al 65 % y el OBC cree que está vacía: según cómo esté programado, entra en modo seguro y apaga la carga útil. La segunda multiplica primero $79000 / 1200$ y da 65: los decimales se siguen tirando, pero al final, donde pesan menos. La tercera mete un **float** en la cuenta ($100.0f$) y C pasa **toda** la operación a punto flotante: 65,8.

LA POSTA

En C el tipo no es un detalle de escritura: decide cuánto entra, qué pasa cuando se pasa, y cómo se hace cada cuenta. Por cada variable, dos preguntas: ¿puede ser negativo? ¿hasta dónde llega? Y en cada división, una tercera: ¿estos dos son enteros?

MÓDULO 3

Constantes y calificadores

AL TERMINAR ESTE MÓDULO VAS A PODER

Leer cualquier literal de C (y no caer en el cero de adelante), ponerle nombre a cada número con `#define` sin que la precedencia te lo cambie, y saber qué promete cada calificador: `const` (esto no se toca), `volatile` (esto cambia por fuera) y `static` (esto vive toda la ejecución, o esto no sale de este archivo).

3.1 Los literales: cómo se escribe un valor

Un *literal* es un valor escrito directamente en el código, como el 42 en `x = 42`; o la 'S' de un modo de vuelo. Parece lo más inocente del lenguaje, y tiene más formas de las que uno espera:

Forma	Se escribe	Ejemplos de la clase
entero decimal	como siempre, sin cero adelante	0, 123
entero octal	con un 0 adelante	0123
entero hexadecimal	con 0x adelante	0xA024
entero binario	con 0b adelante	0b1001
flotante	con punto, o con exponente e	0.6, 2.5e9, -6.3E5
carácter	entre comillas simples	'D'
cadena	entre comillas dobles	"Leandro"

El mismo número, escrito de cuatro maneras, y dos sorpresas de yapa:

UN VALOR, MUCHAS FORMAS DE ESCRIBIRLO – m03-literales.c

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint8_t config_dec = 42;           /* decimal */
    uint8_t config_hex = 0x2A;        /* hexadecimal: 0x adelante */
    uint8_t config_oct = 052;         /* octal: un cero adelante */
    uint8_t config_bin = 0b101010;    /* binario: extension de gcc, estandar recién en C23 */

    uint8_t id_camara = 010;          /* la idea: el ID 10, con dos cifras */

    float altura_m = 5.5e5f;          /* 5,5 x 10^5 m: 550 km */
    char modo = 'S';                  /* S de safe mode */

    printf("Config: %u %u %u %u\n", config_dec, config_hex, config_oct, config_bin);
    printf("Config en hexa: 0x%02X\n", config_dec);
    printf("ID de la camara: %u\n", id_camara);
    printf("Altura: %.0f m\n", altura_m);
    printf("Modo: %c, que para C es el numero %d\n", modo, modo);
    return 0;
}
```

```
SALIDA (copiada de la corrida real)
Config: 42 42 42 42
Config en hexa: 0x2A
ID de la camara: 8
Altura: 550000 m
Modo: S, que para C es el numero 83
```

Las cuatro configuraciones valen 42: la base es cómo lo escribís vos, no cómo lo guarda la máquina (que siempre guarda bits). Para registros y máscaras se usa hexadecimal, porque cada cifra son exactamente cuatro bits y se lee de un vistazo; `%02X` lo imprime así.

Ojo: un cero adelante **no** es prolijidad: convierte el número en octal. El que escribió `010` quería el ID 10 y le salió el 8, que es otra cámara. El `0123` de la diapositiva, de paso, vale 83 en decimal. C no avisa: para el compilador es un número perfectamente válido, sólo que no el que pensabas.

La 'S' es un carácter, y para C un carácter **es** un número: su código ASCII, 83. Por eso se puede imprimir con `%c` o con `%d`, y por eso `char` es también «un entero chico». Y el `-6.3E5` de la tabla no es un literal negativo: es el literal `6.3E5` con el operador menos adelante. Detalle de abogado, pero el día que leas un mensaje de error sobre eso, vas a saber de qué habla.

Ojo: `0b` no es C11: es una extensión de gcc, y el estándar la adopta recién en C23. Con los flags de la cátedra compila sin decir nada; si agregás `-Wpedantic`, gcc avisa que *binary constants are a C2X feature or GCC extension*. En la placa compila igual (el compilador de CubeIDE también es gcc), pero si algún día el código pasa por otro compilador, es lo primero que se cae.

El sufijo dice el tipo del literal. Sin sufijo, `5.5e5` es `double`; con `f`, `float`. En un entero, `u` lo hace sin signo:

Sufijo	Tipo del literal	Ejemplo
ninguno	<code>int</code> (entero) o <code>double</code> (con punto)	<code>1500, 3.3</code>
<code>u</code>	<code>unsigned int</code>	<code>1500u</code>
<code>f</code>	<code>float</code>	<code>3.3f</code>
<code>UL</code>	<code>unsigned long</code>	<code>100000UL</code>

3.2 #define: un nombre para un número

Un `3300` suelto en el medio del código no dice nada: ¿son milivolts, grados, milisegundos? `#define` le pone nombre. Es una orden para el *preprocesador*, el paso que corre **antes** del compilador y que reemplaza texto: donde dice `CELDA_MIN_MV`, escribe `3300u`, y recién ahí compila. No tiene tipo, no ocupa memoria, y no lleva punto y coma (si se lo ponés, el `;` viaja con el reemplazo).

LOS LÍMITES DE UNA CELDA DE LA BATERÍA — `m03-bateria.c`

```
/* C libraries */
#include <stdio.h>
#include <stdint.h>

/* Macros */
#define CELDA_MIN_MV 3300u /* por debajo, la celda se degrada */
#define CELDA_MAX_MV 4200u /* por encima, se arruina */
#define CARGA_MAX_MA 1500u /* corriente de carga maxima */

int main(void) {
    /* Main program */
    const uint16_t celda_mv = 3290u; /* lectura del ADC: una vez tomada, no se toca */
    const uint16_t carga_ma = 1200u;

    uint8_t baja      = celda_mv < CELDA_MIN_MV;
    uint8_t alta      = celda_mv > CELDA_MAX_MV;
    uint8_t carga_ok   = carga_ma <= CARGA_MAX_MA;
    uint8_t celda_ok   = !baja && !alta;

    printf("Celda: %u mV, carga: %u mA\n", celda_mv, carga_ma);
    printf("baja=%u alta=%u carga_ok=%u celda_ok=%u\n", baja, alta, carga_ok, celda_ok);
    printf("Alarmas activas: %u\n", baja + alta + !carga_ok);
    return 0;
}
```

```
SALIDA (copiada de la corrida real)
Celda: 3290 mV, carga: 1200 mA
baja=1 alta=0 carga_ok=1 celda_ok=0
Alarmas activas: 1
```

La convención es mayúsculas, y la unidad en el nombre: `CELDA_MIN_MV` no deja dudas, `LIMITE` sí. Si mañana cambia la batería, se cambia un número en un solo lugar y el resto del programa se entera solo.

LO QUE PIDE LA CÁTEDRA — LO CONFIGURABLE, CON NOMBRE

Lo que se repite va en una función; lo configurable, en macros con nombre. Los límites, los umbrales y los tiempos se definen una vez, arriba, en la sección `Macros` de `template.c`.

3.2.1 La comparación vale 0 o 1

Mirá las cuatro variables del medio: no hay ningún `if`. En C, una comparación es una expresión que **vale** algo: **1** si es cierta, **0** si no. Se puede guardar en un `uint8_t`, combinar con `&&` (y), `||` (o) y `!` (no), e incluso sumar: la última línea cuenta cuántas alarmas hay prendidas sumando condiciones. La celda está a 10 mV del límite, así que `baja` vale 1 y `celda_ok` vale 0: el OBC ya sabe que tiene que cortar el consumo, y no hizo falta un solo `if` para enterarse.

3.3 La trampa del `#define`: los paréntesis

Como el preprocesador reemplaza **texto**, sin entender nada de cuentas, una macro con una operación adentro es una bomba de tiempo:

EL MARGEN QUE SE COMIÓ LA PRECEDENCIA — `m03-macro.c`

```
#include <stdio.h>
#include <stdint.h>

#define MARGEN_MAL    100u + 50u    /* sin parentesis */
#define MARGEN_BIEN  (100u + 50u)  /* con parentesis */

int main(void) {
    /* el doble del margen, en mA: tendria que dar 300 */
    uint16_t doble_mal  = 2u * MARGEN_MAL;
    uint16_t doble_bien = 2u * MARGEN_BIEN;

    printf("Sin parentesis: %u mA\n", doble_mal);
    printf("Con parentesis: %u mA\n", doble_bien);
    return 0;
}
```

```
SALIDA (copiada de la corrida real)
Sin parentesis: 250 mA
Con parentesis: 300 mA
```

`2u * MARGEN_MAL` se convierte, letra por letra, en `2u * 100u + 50u`. La multiplicación va primero, y queda $200 + 50 = 250$. Con paréntesis, en `2u * (100u + 50u)`, que da los 300 que se querían. Cincuenta miliampere de margen que desaparecieron sin un warning, y que alguien va a extrañar el día que el panel entregue menos de lo que dice la hoja de datos.

MEJORA, NO ES DE LA CÁTEDRA — PARÉNTESIS Y SUFIJO, SIEMPRE

Toda macro que tenga una operación va entera entre paréntesis, sin excepción (las macros con parámetros piden más todavía: se ven en el módulo 11). Y toda constante entera sin signo lleva la `u`, como `3300u`: lo exige la regla 7.2 de MISRA-C:2012, el estándar de C para sistemas críticos que vuelve en el módulo 12.

3.4 const: el dato que no se toca

`const` delante del tipo dice que el programa no puede modificar esa variable. En el ejemplo de la batería, la lectura de la celda es `const`: una vez tomada, no hay motivo para cambiarla, y si alguien lo intenta:

```
const uint16_t celda_mv = 3290u;
celda_mv = 3300u;
```

gcc no avisa: **corta**. Es un error, no un warning, y no hay ejecutable: `error: assignment of read-only variable 'celda_mv' (gcc 13.3 de Ubuntu).`

LO QUE PIDE LA CÁTEDRA – CONST

`const` va antes del tipo y hace que el dato no se pueda modificar. Sirve para tres cosas: que no lo cambie el programa, poder ubicar el dato en la memoria flash (de sólo lectura), y documentar la intención de quien lo escribió.

Lo de la flash importa más de lo que parece. La NUCLEO-F446RE tiene 512 KB de flash y 128 KB de RAM: una tabla de calibración de 2 KB declarada `const` (y global, o `static`) se queda en la flash, donde sobra lugar; sin el `const`, el valor inicial igual se guarda en la flash, y además el programa lo copia a la RAM al arrancar, donde no sobra nada. Pagás dos veces por un dato que no ibas a cambiar.

Ojo: la diapositiva declara `const int c1, c2 = 10;`. El `const` alcanza a las dos variables, pero el `= 10` es sólo de `c2`. Si eso está adentro de una función, `c1` queda con lo que haya en la memoria, y como es `const`, **no se puede arreglar después**: una constante de valor desconocido, para siempre. Se declara una por línea y cada una con su valor.

¿`#define` o `const`? Las dos ponen nombre a un valor, pero no son lo mismo:

Pregunta	<code>#define</code>	<code>const</code>
¿tiene tipo, y el compilador lo chequea?	no	sí
¿la ve el depurador?	no: ya no existe	sí
¿ocupa memoria?	no	sí (en flash, si es global)
¿sirve como tamaño de un vector global o en un <code>case</code> ?	sí	no, en C

La regla práctica: los límites y parámetros de configuración, con `#define` (como pide la cátedra); los datos que se calculan o se leen una vez y no cambian más, y las tablas, con `const`.

3.5 volatile: lo que cambia por fuera

El compilador optimiza suponiendo que una variable sólo cambia cuando el código que está viendo la cambia. Si en un bucle nadie escribe `dato_listo`, puede leerla una sola vez, guardarla en un registro del procesador, y no volver a mirar la memoria nunca más. En la PC eso es razonable. En un microcontrolador es un desastre, porque hay cosas que cambian **sin que el código las toque**: un registro del hardware que se pone en 1 cuando el ADC terminó, o una variable que escribe una interrupción (la ISR, la función que el hardware llama cuando pasa algo).

`volatile` le avisa al compilador justamente eso: este dato puede cambiar por fuera del flujo del programa, así que **leelo de la memoria cada vez**.

ESPERAR AL MAGNETÓMETRO, PERO NO PARA SIEMPRE – m03-espera.c

```

/* C libraries */
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

/* Macros */
#define ESPERA_MAX    100000u    /* cota fija: la espera SIEMPRE termina */

/* Global variables */
/* La pone en 1 la interrupcion del magnetometro, no el codigo de abajo */
static volatile uint8_t dato_listo = 0u;

int main(void) {
    /* Main program */
    uint32_t intentos = 0u;

    while ((dato_listo == 0u) && (intentos < ESPERA_MAX)) {
        intentos++;
    }

    if (dato_listo != 0u) {
        printf("Magnetometro: dato listo tras %" PRIu32 " intentos\n", intentos);
    } else {
        printf("Magnetometro: sin respuesta tras %" PRIu32 " intentos\n", intentos);
        printf("Magnetometro marcado como caido\n");
    }
    return 0;
}

```

SALIDA (copiada de la corrida real)
Magnetometro: sin respuesta tras 100000 intentos
Magnetometro marcado como caido

En la placa, `dato_listo` la pondría en 1 la interrupción del magnetómetro. En la PC no hay magnetómetro ni interrupción, así que nadie la toca y la espera vence: cien mil intentos, y el sensor queda marcado como caído. Que es exactamente lo que tiene que pasar si el sensor se murió en órbita. Sin la cota, el OBC se queda esperando a un magnetómetro que no va a contestar, con el resto del satélite congelado detrás, hasta que el watchdog se apiade y lo reinicie.

LO QUE PIDE LA CÁTEDRA – ESPERAS CON COTA Y VOLATILE

Todo bucle de espera lleva una cota fija: tiene que terminar aunque el hardware no conteste. Y lo que cambia por fuera del flujo del programa (hardware, interrupciones) se declara `volatile`, que obliga a leerlo de memoria cada vez.

Ojo: `volatile` no hace que una operación sea indivisible. Si `main` y una ISR hacen `contador++` sobre la misma variable, ese `++` es leer, sumar y escribir: si la interrupción cae justo en el medio, se pierde una cuenta, con `volatile` y todo. Eso se arregla con secciones críticas, en el módulo 12.

3.5.1 const volatile: sólo lectura, y cambia solo

Los dos juntos no se contradicen: `const` dice que **el programa** no lo escribe; `volatile`, que cambia igual, por otro lado. Es la descripción exacta de un registro de estado del hardware:

```

/* Registro de estado del ADC1 de la F446RE: lo escribe el hardware, no nosotros */
#define ADC1_SR    (*(const volatile uint32_t *)0x40012000u)

```

Los asteriscos y el número de dirección son punteros, y se entienden en el módulo 8. Por ahora alcanza con leer las dos palabras: el programa no puede escribir ahí (`const`), y cada lectura va al hardware a buscar el valor de ese instante (`volatile`).

3.6 `static`: dos palabras con el mismo nombre

`static` significa dos cosas distintas según dónde se escriba, lo que es una decisión de diseño de los años setenta que todavía pagamos.

Adentro de una función, cambia cuánto vive la variable. Una variable local común nace en cada llamada y muere al salir; una `static` nace una sola vez, antes de `main`, y conserva su valor entre llamadas:

CONTAR LOS REINICIOS DEL WATCHDOG — `m03-watchdog.c`

```
/* C libraries */
#include <stdio.h>
#include <stdint.h>

/* Global variables */
static uint16_t resets_totales = 0u; /* static global: solo la ve este .c */

/* Functions declaration */
static void registrar_reset(void);

int main(void) {
    /* Main program */
    registrar_reset();
    registrar_reset();
    registrar_reset();
    printf("Resets totales: %u\n", resets_totales);
    return 0;
}

/* Functions definition */
static void registrar_reset(void) {
    uint16_t automatica = 0u; /* nace en cada llamada */
    static uint16_t cuenta = 0u; /* nace una vez, vive toda la ejecucion */

    automatica++;
    cuenta++;
    resets_totales++;
    printf("Reset del watchdog: automatica=%u, static=%u\n", automatica, cuenta);
}
```

```
SALIDA (copiada de la corrida real)
Reset del watchdog: automatica=1, static=1
Reset del watchdog: automatica=1, static=2
Reset del watchdog: automatica=1, static=3
Resets totales: 3
```

`registrar_reset` es una función: se declara arriba de `main` y se define abajo, como pide `template.c` (el detalle es del módulo 6). Cada vez que se la llama, `automatica` vuelve a nacer en cero y llega a 1; `cuenta` arranca donde había quedado.

Ojo: el `= 0u` de una `static` local se ejecuta **una vez**, no en cada llamada. Parece que la reinicia y no: si la reiniciara, no serviría para nada.

Afuera de una función —en una variable global, o delante de una función—, `static` cambia quién la ve: sólo el `.c` donde está escrita. `resets_totales` y `registrar_reset` son `static`, así que otro archivo del proyecto no las puede tocar ni chocar con ellas. Con un archivo da igual; con veinte (el módulo 11) es la diferencia entre que dos archivos tengan cada uno su `contador`, tranquilos, o que el linker corte con *multiple definition*. Y con compiladores viejos, algo peor: que no corte y los dos compartan el mismo `contador` sin que nadie lo sepa.

LO QUE PIDE LA CÁTEDRA – STATIC

`static` local: conserva su valor entre llamadas y vive toda la ejecución. `static` global o en una función: visibilidad limitada al `.c`, para encapsular y evitar choques de nombres. Y en vuelo, sin `malloc`: la memoria se reserva así, de entrada y con tamaño fijo.

LA POSTA

Cuatro palabras, cuatro preguntas. ¿Es un número que no cambia nunca y tiene que tener nombre? `#define`, con paréntesis y sufijo. ¿Es un dato que el programa no debe tocar? `const`. ¿Puede cambiar sin que tu código lo toque? `volatile`, y la espera con cota. ¿Tiene que acordarse de algo entre llamadas, o no debe salir de este archivo? `static`.

MÓDULO 4

Operadores, y los de bit

AL TERMINAR ESTE MÓDULO VAS A PODER

Hacer cuentas enteras sin perder nada en el camino (/ y % juntos), saber qué vale una condición y cuándo C ni se molesta en evaluarla, leer una expresión en el orden en que la lee el compilador, y manejar bits sueltos: prender, apagar, conmutar y consultar uno sin tocar a los vecinos, y armar y desarmar un campo adentro de una palabra.

4.1 Los aritméticos, y el resto

Los de toda la vida: +, -, *, /, y uno que en la secundaria no se usa nunca y en un micro se usa todo el tiempo: %, el *resto* de la división entera. $17 / 5$ vale 3 (el módulo 2 ya avisó que la división entera tira los decimales) y $17 \% 5$ vale 2, lo que sobró. Juntos no pierden nada: $17 = 3 \cdot 5 + 2$.

Ese par es justo lo que hace falta para pasar el tiempo de misión, que el OBC lleva en milisegundos desde el despegue, a algo que un humano pueda leer:

EL TIEMPO DE MISIÓN, EN DÍAS, HORAS, MINUTOS Y SEGUNDOS — m04-reloj.c

```
/* C libraries */
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

/* Macros */
#define MS_POR_SEG      1000u
#define MS_POR_MIN      (60u * MS_POR_SEG)
#define MS_POR_HORA     (60u * MS_POR_MIN)
#define MS_POR_DIA      (24u * MS_POR_HORA)

int main(void) {
    /* Main program */
    const uint32_t met_ms = 93784567u; /* tiempo de mision: ms desde el despegue */
    uint32_t resto = met_ms;

    uint32_t dias  = resto / MS_POR_DIA;  resto %= MS_POR_DIA;
    uint32_t horas = resto / MS_POR_HORA;  resto %= MS_POR_HORA;
    uint32_t mins  = resto / MS_POR_MIN;   resto %= MS_POR_MIN;
    uint32_t segs  = resto / MS_POR_SEG;   resto %= MS_POR_SEG;

    printf("MET %" PRIu32 " ms\n", met_ms);
    printf(" = %" PRIu32 " d %02" PRIu32 ":%02" PRIu32 ":%02" PRIu32 ".%03" PRIu32 "\n",
           dias, horas, mins, segs, resto);
    return 0;
}
```

SALIDA (copiada de la corrida real)
 MET 93784567 ms
 = 1 d 02:03:04.567

Cada renglón hace lo mismo: la división dice cuántas unidades enteras entran, y el resto es lo que queda para la unidad siguiente. Noventa y tres millones de milisegundos son un día, dos horas y monedas: dicho así, el operador de tierra deja de hacer cuentas en una servilleta. Y fijate que las macros tienen paréntesis: es la lección del módulo 3, y acá `MS_POR_DIA` es una cuenta de cuatro factores esperando que alguien la meta en una expresión.

`resto %= MS_POR_DIA` es una *asignación compuesta*: abrevia `resto = resto % MS_POR_DIA`. Existe para casi todos los operadores (+, -, *, /, %, y los de bit que vienen después: &, |, ^, <=<, >=>), y se lee «a la variable de la izquierda, aplícale esto».

Ojo: con negativos, % sigue al dividendo: `-7 % 3` vale `-1`, no `2` (desde C99 la división redondea hacia el cero, y el resto hereda el signo). Si lo usás para saber «en qué lugar de una vuelta estoy» con un número que puede ser negativo, el resultado puede ser negativo también. Otro argumento para el `unsigned` por defecto. Y % es sólo para enteros: con `float` no compila.

4.2 Relacionales y lógicos

El módulo 3 adelantó que una comparación **vale 1** o **0**. Éstos son todos:

Relacional	Vale 1 si...	Lógico	Vale 1 si...
<code>a == b</code>	son iguales	<code>p && q</code>	las dos son distintas de cero (y)
<code>a != b</code>	son distintos	<code>p q</code>	alguna es distinta de cero (o)
<code>a < b</code> , <code>a <= b</code>	menor, menor o igual	<code>!p</code>	<code>p</code> vale cero (no)
<code>a > b</code> , <code>a >= b</code>	mayor, mayor o igual		

Y tienen una propiedad que parece un detalle y es una herramienta: `&&` y `||` evalúan de izquierda a derecha y **cortan apenas saben la respuesta**. Si lo de la izquierda de un `&&` vale cero, el resultado ya es cero, y lo de la derecha no se ejecuta. Es el *cortocircuito*, y sirve de guardia:

TEMPERATURA DE LA CÁMARA, CON GUARDIA Y MODO — `m04-logicos.c`

```
/* C libraries */
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

/* Macros */
#define TEMP_MAX_DC 450 /* 45.0 grados, en decimas: la temperatura puede ser negativa */

int main(void) {
    /* Main program */
    uint16_t muestras = 0u; /* el sensor de la camara todavia no midio nada */
    int32_t suma_dc = 0;

    /* Si muestras vale 0, la division ni se intenta: && corta antes */
    uint8_t caliente = (muestras != 0u) && ((suma_dc / muestras) > TEMP_MAX_DC);
    printf("Sin muestras: caliente=%u\n", caliente);

    suma_dc += 412; muestras++;
    suma_dc += 497; muestras++;
    suma_dc += 516; muestras++;
    caliente = (muestras != 0u) && ((suma_dc / muestras) > TEMP_MAX_DC);
    printf("Promedio %" PRIu32 " dC en %u muestras: caliente=%u\n",
           suma_dc / muestras, muestras, caliente);

    char modo = caliente ? 'S' : 'N'; /* S: modo seguro, N: nominal */
    printf("Modo: %c\n", modo);

    uint8_t a = 2u;
    uint8_t b = 1u;
    printf("a & b = %u, a && b = %u\n", a & b, a && b);

    uint8_t n = 5u;
    uint8_t antes = n++;
    uint8_t despues = ++n;
    printf("antes=%u despues=%u n=%u\n", antes, despues, n);
    return 0;
}
```

```

SALIDA (copiada de la corrida real)
Sin muestras: caliente=0
Promedio 475 dC en 3 muestras: caliente=1
Modo: S
a & b = 0, a && b = 1
antes=5 despues=7 n=7

```

Con cero muestras, (`muestras != 0u`) vale 0 y la división `suma_dc / muestras` **no se hace**. Sin esa guardia, en la PC el programa muere con *Floating point exception* (medido: el sistema lo mata, y no imprime nada más). En la placa es peor, porque no muere: dividir por cero es comportamiento indefinido para C, y en un Cortex-M4 como el de la F446RE la instrucción de dividir, de fábrica, devuelve 0 y sigue de largo (salvo que se prenda la trampa `DIV_0_TRP`, según el manual de arquitectura de ARM). El OBC calcula un promedio de cero décimas, decide que la cámara está fresquita, y sigue tan campante.

Con tres muestras el promedio da 475 décimas, 47,5 grados, por encima del límite: la cámara está caliente y el modo pasa a seguro.

Ojo: `&` y `&&` no son lo mismo, por más que se lean parecido. La penúltima línea lo muestra: `2 & 1` vale 0 (opera bit a bit: 10 y 01 no tienen ningún 1 en el mismo lugar), y `2 && 1` vale 1 (los dos son distintos de cero). Se escribe uno por el otro, compila igual, y el `if` decide al revés.

MEJORA, NO ES DE LA CÁTEDRA – NADA IMPORTANTE A LA DERECHA DE UN `&&` O UN `||`

Como lo de la derecha puede no ejecutarse, ahí no va nada que cambie algo: ni un `++`, ni una asignación, ni una función que escriba un registro. En `(listo != 0u) && (reintentos++ < 3u)`, el contador sólo cuenta cuando `listo` vale distinto de cero, y eso nadie lo quiso. Es la regla 13.5 de MISRA-C:2012.

4.2.1 `++` y `--`, antes o después

`n++` suma uno a `n`, igual que `n += 1u`. La diferencia aparece cuando se usa el **valor** de la expresión, como en las dos últimas líneas del ejemplo: `antes = n++` guarda el valor viejo y después suma (`antes` vale 5, `n` pasa a 6); `despues = ++n` suma primero y guarda el nuevo (`n` y `despues` valen 7).

Ojo: `k = k++`; no es una forma rebuscada de no hacer nada: es comportamiento indefinido, porque la misma variable se modifica dos veces sin un orden establecido. gcc avisa con `-Wall: operation on 'k' may be undefined` (gcc 13.3, medido). La regla práctica: un `++` por variable y por expresión, y mejor si va solo en su renglón.

4.3 El ternario

`condición ? valor_si : valor_no` es un `if` que **vale algo**, y por eso puede ir a la derecha de un `=`. En el ejemplo, `modo` queda en 'S' si la cámara está caliente y en 'N' si no, en un renglón y sin repetir el nombre de la variable en dos ramas.

Ojo: un ternario adentro de otro compila perfecto y no lo entiende nadie, incluido quien lo escribió, a la semana. Uno por expresión, para elegir un valor; si hay que elegir qué **hacer**, es un `if`.

4.4 Quién va primero: la precedencia

Una expresión con varios operadores se evalúa en un orden fijo, y no es el que uno supone mirando la línea. De mayor a menor, los de este módulo:

Operadores	Notas
! ~ ++ -- y el menos de un solo operando	los de un operando van primero
* / %	
+ -	
<< >>	
< <= > >=	
== !=	antes que los de bit: ahí está la trampa
&	
^	
&&	
? :	
= += -= &= = y el resto	van último: primero se calcula, después se guarda

La fila marcada es el tropiezo clásico al consultar un bit:

¿EL GPS TIENE POSICIÓN? – m04-precedencia.c

```

/* C libraries */
#include <stdio.h>
#include <stdint.h>

/* Macros */
#define ESTADO_GPS_FIX    0x08u    /* bit 3: el GPS tiene posicion */

int main(void) {
    /* Main program */
    uint8_t estado = 0x05u;    /* bits 0 y 2 prendidos; el 3, apagado: sin fix */

    if (estado & ESTADO_GPS_FIX == ESTADO_GPS_FIX) {
        printf("sin parentesis: GPS con fix\n");
    } else {
        printf("sin parentesis: GPS sin fix\n");
    }

    if ((estado & ESTADO_GPS_FIX) == ESTADO_GPS_FIX) {
        printf("con parentesis: GPS con fix\n");
    } else {
        printf("con parentesis: GPS sin fix\n");
    }
    return 0;
}

```

SALIDA (copiada de la corrida real)

```

sin parentesis: GPS con fix
con parentesis: GPS sin fix

```

```

LO QUE DICE GCC (copiado de la corrida real)
m04-precedencia.c: In function 'main':
m04-precedencia.c:12:20: warning: suggest parentheses around comparison in operand of '&' [-Wparentheses]
    12 |         if (estado & ESTADO_GPS_FIX == ESTADO_GPS_FIX) {
        |                ^

```

Como `==` va antes que `&`, la primera condición se lee `estado & (0x08u == 0x08u)`, o sea `estado & 1`: pregunta por el bit 0, que está prendido, y responde que el GPS tiene posición cuando no la tiene. Un OBC que cree saber dónde está es más peligroso que uno que sabe que no sabe. gcc lo ve venir y avisa con `-Wall`; por eso la cátedra exige cero warnings, y por eso éste no se ignora.

MEJORA, NO ES DE LA CÁTEDRA – LOS PARÉNTESIS NO CUESTAN NADA

Cuando una expresión mezcla operadores de familias distintas (aritméticos, de bit, de comparación, lógicos), se ponen paréntesis aunque la precedencia dé bien: el que lee no tiene por qué saberse la tabla de memoria. Es la regla 12.1 de MISRA-C:2012, y el ejemplo de arriba es su razón de existir.

4.5 Los operadores de bit

Hasta acá, un `uint8_t` era un número del 0 al 255. Ahora es otra cosa: **ocho interruptores** en fila. En un micro se usa así todo el tiempo, porque cada registro del hardware es eso: en uno del GPIO, cada bit es una pata; en uno de estado, cada bit es un aviso. Los operadores trabajan bit por bit, cada uno con su vecino del mismo lugar:

Operador	Cada bit del resultado vale 1 si...	Ejemplo en binario
<code>a & b</code>	los dos valen 1 (y)	<code>1100 & 1010</code> da <code>1000</code>
<code>a b</code>	alguno vale 1 (o)	<code>1100 1010</code> da <code>1110</code>
<code>a ^ b</code>	son distintos (o exclusivo)	<code>1100 ^ 1010</code> da <code>0110</code>
<code>~a</code>	el de <code>a</code> vale 0 (los da vuelta)	<code>~1100</code> da <code>0011</code> (en 4 bits)
<code>a << n</code>	corre todo <code>n</code> lugares a la izquierda; entran ceros	<code>0011 << 2</code> da <code>1100</code>
<code>a >> n</code>	corre todo <code>n</code> lugares a la derecha	<code>1100 >> 2</code> da <code>0011</code>

LO QUE PIDE LA CÁTEDRA – BITS CON OPERADORES

Los bits se manejan con los operadores de bit (máscaras, `|=`, `&=`, `~`, `^`, `<<`, `>>`), sobre tipos sin signo de ancho fijo.

4.5.1 Las cuatro operaciones con una máscara

Una *máscara* es un número con unos sólo en los bits que te interesan. `(1u << 3)` es un 1 corrido tres lugares: `00001000`, el bit 3 y nada más. Con una máscara, las cuatro cosas que se le hacen a un bit son siempre las mismas:

Para...	Se escribe	Por qué anda
prender	<code>x = M;</code>	<code> 1</code> pone un 1; <code> 0</code> deja lo que había
apagar	<code>x &= ~M;</code>	<code>~M</code> tiene ceros sólo en los bits de <code>M</code> ; <code>& 0</code> borra, <code>& 1</code> deja
conmutar	<code>x ^= M;</code>	<code>^ 1</code> da vuelta el bit; <code>^ 0</code> lo deja
consultar	<code>(x & M) != 0u</code>	queda distinto de cero sólo si el bit estaba en 1

El OBC tiene un byte que dice qué está alimentado, un bit por equipo:

PRENDER Y APAGAR EQUIPOS, DE A UN BIT — m04-banderas.c

```

/* C libraries */
#include <stdio.h>
#include <stdint.h>

/* Macros */
#define PWR_GPS      (1u << 0)
#define PWR_MAG      (1u << 1) /* magnetometro */
#define PWR_TX       (1u << 2) /* transmisor de radio */
#define PWR_CALEF     (1u << 3) /* calefactor de la bateria */
#define PWR_CAMARA    (1u << 4)

/* Functions declaration */
static void imprimir_bits(const char *paso, uint8_t valor);

int main(void) {
    /* Main program */
    uint8_t potencia = PWR_GPS | PWR_MAG; /* lo que arranca prendido */
    imprimir_bits("arranque", potencia);

    potencia |= PWR_TX | PWR_CAMARA; /* prender dos, sin tocar el resto */
    imprimir_bits("+TX +camara", potencia);

    potencia &= ~PWR_CAMARA; /* apagar uno, sin tocar el resto */
    imprimir_bits("-camara", potencia);

    potencia ^= PWR_CALEF; /* conmutar: si estaba apagado, prende */
    imprimir_bits("^calefactor", potencia);
    potencia ^= PWR_CALEF; /* y otra vez: vuelve a como estaba */
    imprimir_bits("^calefactor", potencia);

    printf("TX prendido? %u  camara prendida? %u\n",
        (potencia & PWR_TX) != 0u, (potencia & PWR_CAMARA) != 0u);
    return 0;
}

/* Functions definition */
static void imprimir_bits(const char *paso, uint8_t valor) {
    printf("%-12s 0x%02X ", paso, valor);
    for (uint8_t mascara = 0x80u; mascara != 0u; mascara >>= 1) {
        putchar(((valor & mascara) != 0u) ? '1' : '0');
    }
    putchar('\n');
}

```

SALIDA (copiada de la corrida real)

```

arranque      0x03  00000011
+TX +camara   0x17  00010111
-camara       0x07  00000111
^calefactor   0x0F  00001111
^calefactor   0x07  00000111
TX prendido? 1  camara prendida? 0

```

Cada operación toca sólo los bits de su máscara, y el resto del byte pasa intacto: apagar la cámara no apaga el transmisor, que es la diferencia entre una foto menos y un satélite mudo. El ^= dos veces seguidas deja todo como estaba, que es la gracia del o exclusivo (y la razón por la que conmutar a ciegas es mala idea: si no sabés cómo estaba, no sabés cómo queda).

La función `imprimir_bits` es la forma de ver un número en binario: una máscara arranca en `0x80` (el bit 7) y se corre un lugar a la derecha por vuelta, consultando cada bit; cuando el 1 se cae por la derecha, la máscara

vale cero y se termina. El `for` es del módulo 5, y el `const char *`, de los módulos 7 y 8: acá alcanza con leer lo que hace.

La idea: C11 no tiene un especificador para binario en `printf`. C23 agrega `%b`, y la glibc de Ubuntu 24.04 ya lo entiende (medido: imprime `101` para un `5`, y gcc 13 no dice nada con los flags de la cátedra). Pero no es C11, y en la biblioteca de C de la placa no está medido: la función de tres renglones anda en todos lados.

Ojo: `~` sobre un `uint8_t` no da un `uint8_t`. Antes de operar, C promueve el valor a `int`: con `m = 0x04u`, `~m` no es `0xFB` sino `0xFFFFFBB`, y `~m == 0xFBu` **nunca** es cierto. gcc avisa con `-Wextra: comparison of promoted bitwise complement of an unsigned value with constant` (gcc 13.3, medido). En `x &= ~M` no hace daño, porque el resultado se recorta al guardarlo en `x`; para comparar, se recorta a mano: `(uint8_t)~m == 0xFBu`.

4.5.2 Correr bits: armar y desarmar una palabra

Los protocolos meten varios datos en una misma palabra, cada uno en su rango de bits. La cabecera de un paquete de telemetría es el caso de libro: el estándar CCSDS de paquetes espaciales (CCSDS 133.0-B), el que usan casi todas las misiones, arranca cada paquete con 16 bits repartidos en cuatro campos: versión (3 bits), tipo (1: telemetría o telecomando), si hay cabecera secundaria (1) y el *APID* (11), el número que dice de qué subsistema viene el paquete.

LA CABECERA DE UN PAQUETE CCSDS, DE IDA Y DE VUELTA — `m04-paquete.c`

```
/* C libraries */
#include <stdio.h>
#include <stdint.h>

/* Macros */
#define CCSDS_TIPO_TM      0u      /* 0: telemetría, 1: telecomando */
#define APID_TERMICO      0x2C1u   /* el subsistema termico de nuestro OBC */
#define APID_MASCARA      0x07FFu  /* 11 bits */
#define CUENTA_MASCARA    0x3FFFu  /* 14 bits */

int main(void) {
    /* Main program */
    /* Armar: version (3 bits) | tipo (1) | cabecera secundaria (1) | APID (11) */
    uint16_t armada = (uint16_t)((0u << 13) | (CCSDS_TIPO_TM << 12) | (1u << 11)
                                | (APID_TERMICO & APID_MASCARA));
    printf("Armada: 0x%04X\n", armada);

    /* Desarmar una que llego de tierra */
    const uint16_t recibida = 0x1A5Fu;
    printf("Recibida: 0x%04X -> version=%u tipo=%u sec=%u apid=0x%03X\n", recibida,
           (recibida >> 13) & 0x7u, (recibida >> 12) & 0x1u,
           (recibida >> 11) & 0x1u, recibida & APID_MASCARA);

    /* El contador de secuencia da la vuelta solo, con la mascara */
    uint16_t cuenta = 16382u;
    for (uint8_t i = 0u; i < 3u; i++) {
        cuenta = (cuenta + 1u) & CUENTA_MASCARA;
        printf("cuenta=%u\n", cuenta);
    }
    return 0;
}
```

```
SALIDA (copiada de la corrida real)
Armada: 0x0AC1
Recibida: 0x1A5F -> version=0 tipo=1 sec=1 apid=0x25F
cuenta=16383
cuenta=0
cuenta=1
```

Armar es correr cada campo a su lugar con `<<` y juntarlos con `|`: como los rangos no se pisan, el `|` los apila sin mezclarlos. La máscara sobre el APID es cinturón y tirador: si alguien define un APID de más de 11 bits, se recorta en vez de pisar el bit de la cabecera secundaria. El `(uint16_t)` adelante es una conversión explícita (un *cast*): la cuenta se hace en `unsigned int`, de 32 bits, y el *cast* dice que quedarse con 16 es a propósito.

Desarmar es lo inverso: correr a la derecha hasta que el campo quede abajo de todo, y quedarse con sus bits con una máscara. `0x1A5F` resulta ser un telecomando, con cabecera secundaria, para el APID `0x25F`.

El contador de secuencia del paquete tiene 14 bits, así que después de 16383 viene el 0. `& 0x3FFFu` hace eso solo: para un sin signo y una potencia de dos, quedarse con los bits de abajo es lo mismo que el resto de dividir por esa potencia (acá, `% 16384u`). Se escribe con la máscara porque dice lo que es: el campo tiene 14 bits, ni uno más.

Ojo: se corre siempre sobre sin signo. `1 << 31` en un `int` de 32 bits es comportamiento indefinido (el 1 cae en el bit de signo), y gcc no dice nada con los flags de la cátedra (medido). `>>` sobre un negativo depende del compilador (gcc copia el bit de signo: `-16 >> 2` da `-4`). Y correr tantos lugares como bits tiene el tipo, o más, es indefinido siempre. Por eso las máscaras se escriben `1u << n`, con la *u*.

MEJORA, NO ES DE LA CÁTEDRA – BITS, SÓLO SOBRE SIN SIGNO

Los operadores de bit van sobre operandos sin signo (regla 10.1 de MISRA-C:2012), y lo que se corre con `<<` o `>>` va de cero hasta uno menos que el ancho del tipo (regla 12.2). Con el `unsigned` por defecto de la cátedra, la primera viene de regalo.

LA POSTA

`/` y `%` van juntos: uno dice cuántas veces entra, el otro qué sobra. `&&` y `||` cortan apenas saben, y eso se usa de guardia. La precedencia no se adivina: ante la duda, paréntesis, y si mezclás `&` con `==`, paréntesis sin duda. Y un byte son ocho interruptores: `|=` prende, `&= ~` apaga, `^=` conmuta, `&` consulta, `<<` arma y `>>` desarma. Siempre sobre sin signo.